1. Setting up and Installing Hadoop in its two operating modes: • Pseudo distributed, • Fully distributed.

PSEUDO DISTRIBUTED MODE

Configuration Files:

core-site.xml

```
<configuration>
<property>
<name>fs.defaultFS</name>
<value>hdfs://localhost:9000</value>
</property>
</configuration>
```

hdfs-site.xml

```
<configuration>
<property>
<name>dfs.replication</name>
<value>1</value>
</property>
</configuration>
```

mapred-site.xml

```
<configuration>
<property>
<name>mapreduce.framework.name</name>
<value>yarn</value>
</property>
</configuration>
```

yarn-site.xml

```
<configuration>
<property>
<name>yarn.nodemanager.aux-services</name>
<value>mapreduce_shuffle</value>
</property>
</configuration>
```

Commands :

```
:: Format Namenode
hdfs namenode -format

:: Start Hadoop services
start-dfs.cmd
start-yarn.cmd

:: Check running services
jps
```

Output:

```
NameNode
DataNode
```

ResourceManager
NodeManager

FULLY DISTRIBUTED MODE

Configuration Changes

core-site.xml

<value>hdfs://master:9000</value>

hdfs-site.xml

<property>
<name>dfs.replication</name>
<value>3</value>
</property>

workers (or slaves file)

slave1
slave2
slave3

Commands

start-dfs.sh
start-yarn.sh

Output

Master node running NameNode
Slave nodes running DataNode
Cluster working successfully

2.Implementation of the following file management tasks in Hadoop: • Adding files and directories • Retrieving files • Deleting files

Adding Files and Directories

```
cd C:\hadoop-lab\FileManagement

:: Create directory in HDFS
hdfs dfs -mkdir /labdata

:: Upload file to HDFS
hdfs dfs -put sample.txt /labdata/
```

output:

Directory /labdata created successfully
File sample.txt uploaded to HDFS

Retrieving Files

```
:: Display file content
hdfs dfs -cat /labdata/sample.txt

:: Download file from HDFS to local
hdfs dfs -get /labdata/sample.txt C:\hadoop-lab\
```

Output :

Welcome to Hadoop Lab
This is File Management Program

File downloaded successfully

Deleting Files and Directories:

```
:: Delete file
hdfs dfs -rm /labdata/sample.txt

:: Delete directory
hdfs dfs -rmdir /labdata
```

Output:

Deleted /labdata/sample.txt
Removed directory /labdata

3. **Implementation of Word Count Map Reduce program • Find the number of occurrence of each word appearing in the input file(s) • Performing a MapReduce Job for word search count (look for specific keywords in a file)**

Input.txt

Hello Hadoop
Hello Big Data
Hadoop is powerful
Big Data Hadoop

WORD COUNT PROGRAM:

Mapper.py

```
import sys

for line in sys.stdin:
    words = line.strip().split()
    for word in words:
        word = word.lower()
        print(f"{word}\t1")
```

Reducer.py

```python
import sys

current_word = None
current_count = 0

for line in sys.stdin:
    word, count = line.strip().split("\t")
    count = int(count)

    if word == current_word:
        current_count += count
    else:
        if current_word:
            print(f"{current_word}\t{current_count}")
        current_word = word
        current_count = count

if current_word:
    print(f"{current_word}\t{current_count}")
```

Commands:

```bash
cd C:\hadoop-lab\WordCount

:: Create HDFS directory
hdfs dfs -mkdir /wordcount

:: Upload input file
hdfs dfs -put input.txt /wordcount/

:: Run MapReduce Job
hadoop jar %HADOOP_HOME%\share\hadoop\tools\lib\hadoop-streaming*.jar ^
-input /wordcount/input.txt ^
-output /wordcount/output ^
-mapper "python mapper.py" ^
-reducer "python reducer.py"

:: View output
hdfs dfs -cat /wordcount/output/part-00000
```

Output:

big 2
data 2
hadoop 3
hello 2
is 1
powerful 1

KEYWORD SEARCH COUNT

Mapper.py:

```python
import sys

keyword = "hadoop"

for line in sys.stdin:
    words = line.strip().lower().split()
    for word in words:
        if word == keyword:
            print(f"{keyword}\t1")
```

Reducer.py:

```python
import sys

count = 0

for line in sys.stdin:
    word, value = line.strip().split("\t")
    count += int(value)

print(f"{word}\t{count}")
```

Commands:

```
:: Run Keyword Search Job
hadoop jar %HADOOP_HOME%\share\hadoop\tools\lib\hadoop-streaming*.jar ^
-input /wordcount/input.txt ^
-output /wordcount/keyword_output ^
-mapper "python keyword_mapper.py" ^
-reducer "python keyword_reducer.py"

:: View output
hdfs dfs -cat /wordcount/keyword_output/part-00000
```

Output:

hadoop 3

4. **Map Reduce Program for Stop word elimination: • Map Reduce program to eliminate stop words from a large text file.**

   Input.txt
   This is the big data lab and Hadoop is very powerful and useful for processing large data
   Big data is the future and Hadoop is one of the best tools in big data analytics
   In the world of data science and machine learning data plays a very important role
   Hadoop and Spark are widely used in big data processing and data engineering

   Mapper.py

```python
import sys
import string

# Define stop words
stop_words = {
    "is", "the", "and", "a", "an", "in", "on", "at",
    "to", "for", "of", "with", "this", "that", "very"
}

for line in sys.stdin:
    # Remove punctuation
    line = line.strip().translate(str.maketrans('', '', string.punctuation))

    words = line.split()

    for word in words:
        word = word.lower()

        if word not in stop_words:
            print(f"{word}\t1")
```

Reducer.py

```python
import sys

current_word = None
count = 0

for line in sys.stdin:
    word, value = line.strip().split("\t")
    value = int(value)

    if word == current_word:
        count += value
    else:
        if current_word:
            print(current_word)
        current_word = word
        count = value

# Last word
if current_word:
    print(current_word)
```

Command:

```
cd C:\hadoop-lab\StopWord

:: Create HDFS directory
hdfs dfs -mkdir /stopword

:: Upload input file
hdfs dfs -put input.txt /stopword/

:: Run MapReduce Job
hadoop jar %HADOOP_HOME%\share\hadoop\tools\lib\hadoop-streaming*.jar ^
-input /stopword/input.txt ^
-output /stopword/output ^
-mapper "python mapper.py" ^
-reducer "python reducer.py"

:: View output
hdfs dfs -cat /stopword/output/part-00000
```

Output:

big
data
lab
hadoop
powerful
useful

5. **Map Reduce program that mines weather data. Weather sensors collecting data every hour at many locations across the globe gather large volume of log data, which is a good candidate for analysis with MapReduce, since it is semi structured and record-oriented. Data available at: https://github.com/tomwhite/hadoop- book/tree/master/input/ncdc/all. • Find average, max and min temperature for each year in NCDC data set? • Filter the readings of a set based on value of the measurement, Output the line of input files associated with a temperature value greater than 30.0 and store it in a separate file.**

   Input.txt:
   2020,30
   2020,35
   2020,25
   2021,40
   2021,38
   2021,28
   2022,45
   2022,20
   2022,33

   AVG, MAX, MIN TEMPERATURE PER YEAR :
   Mapper.py:

```python
import sys

for line in sys.stdin:
    data = line.strip().split(",")

    if len(data) == 2:
        year = data[0]
        temp = data[1]

        print(f"{year}\t{temp}")
```

Reducer.py:

```
import sys

current_year = None
temps = []

for line in sys.stdin:
year, temp = line.strip().split("\t")
temp = float(temp)

if current_year == year:
temps.append(temp)
else:
if current_year:
avg = sum(temps) / len(temps)
print(f"{current_year}\tAvg:{avg:.2f} Max:{max(temps)} Min:{min(temps)}")

current_year = year
temps = [temp]

# Last year
if current_year:
avg = sum(temps) / len(temps)
print(f"{current_year}\tAvg:{avg:.2f} Max:{max(temps)} Min:{min(temps)}")
```

Commands:

```
hadoop jar %HADOOP_HOME%\share\hadoop\tools\lib\hadoop-streaming*.jar ^
-input /weather/input.txt ^
-output /weather/filter_output ^
-mapper "python filter_mapper.py" ^
-reducer "python filter_reducer.py"

hdfs dfs -cat /weather/filter_output/part-00000
```

Output:

2020 Avg:30.00 Max:35.0 Min:25.0
2021 Avg:35.33 Max:40.0 Min:28.0
2022 Avg:32.67 Max:45.0 Min:20.0

FILTER TEMPERATURE > 30°C

Mapper.py:

```python
import sys

for line in sys.stdin:
    data = line.strip().split(",")

    if len(data) == 2:
        year = data[0]
        temp = float(data[1])

        if temp > 30.0:
            print(line.strip())
```

Reducer.py:

```
import sys

for line in sys.stdin:
print(line.strip())
```

command:

```
hadoop jar %HADOOP_HOME%\share\hadoop\tools\lib\hadoop-streaming*.jar ^
-input /weather/input.txt ^
-output /weather/filter_output ^
-mapper "python filter_mapper.py" ^
-reducer "python filter_reducer.py"

hdfs dfs -cat /weather/filter_output/part-00000
```

output:

2020,35
2021,40
2021,38
2022,45
2022,33

6. Install and Run Pig then write Pig Latin scripts to sort, group, join, project, and filter your data.

Emp.txt:

1,Ram,HR,20000
2,Shyam,IT,30000
3,Ravi,IT,25000
4,Sita,HR,22000
5,Gita,Finance,28000

Dept.txt:

HR,HumanResource
IT,InformationTech
Finance,Accounts

Start Pig :

```
cd C:\pig\bin
pig -x local
```

load Data :

```
emp = LOAD 'C:/hadoop-lab/Pig/emp.txt'
USING PigStorage(',')
AS (id:int, name:chararray, dept:chararray, salary:int);

dept = LOAD 'C:/hadoop-lab/Pig/dept.txt'
USING PigStorage(',')
AS (dept:chararray, dept_name:chararray);
```

Projection:

```
proj = FOREACH emp GENERATE name, salary;
DUMP proj;
```

output:

```
(Ram,20000)
(Shyam,30000)
(Ravi,25000)
(Sita,22000)
(Gita,28000)
```

Filtering data:

```
high_salary = FILTER emp BY salary > 25000;
DUMP high_salary;
```

output:

```
(2,Shyam,IT,30000)
(5,Gita,Finance,28000)
```

Grouping Data:

```
grp = GROUP emp BY dept;
count_emp = FOREACH grp GENERATE group, COUNT(emp);
DUMP count_emp;
```

output:

(HR,2)

(IT,2)

(Finance,1)

Storing data:

```
sorted = ORDER emp BY salary DESC;

DUMP sorted;
```

Output:

```
(2,Shyam,IT,30000)
```

(5,Gita,Finance,28000)

(3,Ravi,IT,25000)

(4,Sita,HR,22000)

(1,Ram,HR,20000)

Join operation:

joined = JOIN emp BY dept, dept BY dept;

DUMP joined;

Output:

(1,Ram,HR,20000,HR,HumanResource)

(4,Sita,HR,22000,HR,HumanResource)

(2,Shyam,IT,30000,IT,InformationTech)

(3,Ravi,IT,25000,IT,InformationTech)

(5,Gita,Finance,28000,Finance,Accounts)

7. Write a Pig Latin script for finding TF-IDF value for book dataset (A corpus of eBooks available at: Project Gutenberg)

Book1.txt

Hadoop is big data framework Hadoop is powerful

Book2.txt

Big data analytics and Hadoop are important

Book3.txt

Data science uses big data and machine learning

Start pig:

cd C:\pig\bin

pig -x local

Pig Latin Script (tfidf.pig):

```
-- Load all books
docs = LOAD 'C:/hadoop-lab/TFIDF/books'
USING TextLoader() AS (line:chararray);

-- Tokenize words
words = FOREACH docs GENERATE FLATTEN(TOKENIZE(LOWER(line))) AS word;

-- Remove empty words
clean_words = FILTER words BY word IS NOT NULL;
```

```
-- Term Frequency (TF)
grp_words = GROUP clean_words BY word;
tf = FOREACH grp_words GENERATE group AS word, COUNT(clean_words) AS term_count;

-- Total words count
total_words = FOREACH (GROUP clean_words ALL)
GENERATE COUNT(clean_words) AS total;

-- Cross TF with total words
tf_final = FOREACH tf GENERATE
word,
(term_count / (double)total_words.total) AS tf;

-- Document Frequency (DF)
doc_words = FOREACH docs GENERATE
FLATTEN(TOKENIZE(LOWER(line))) AS word, line;

grp_doc = GROUP doc_words BY word;
df = FOREACH grp_doc GENERATE group AS word, COUNT(doc_words.line) AS doc_count;

-- Total documents
total_docs = FOREACH (GROUP docs ALL)
GENERATE COUNT(docs) AS total_docs;

-- Join TF and DF
joined = JOIN tf_final BY word, df BY word;

-- Compute TF-IDF
tfidf = FOREACH joined GENERATE
tf_final::word AS word,
tf_final::tf * LOG((double)total_docs.total_docs / df::doc_count) AS tfidf_value;

DUMP tfidf;
```

Run Script:

```
cd C:\hadoop-lab\TFIDF

pig -x local tfidf.pig
```

Output:

(hadoop,0.12)

(big,0.10)

(data,0.15)

(analytics,0.08)

(science,0.09)

(machine,0.07)

(learning,0.07)

8. Install and Run Hive then use Hive to create, alter, and drop databases, tables, views, functions, and indexes

Students.txt:

1,Ram,20
2,Shyam,22
3,Ravi,21

Start hive :

cd C:\hive\bin
hive

Create database

```
CREATE DATABASE labdb;
SHOW DATABASES;
```

Output:
default
labdb

```
USE labdb;

CREATE TABLE student (
   id INT,
   name STRING,
   age INT
)
ROW FORMAT DELIMITED
FIELDS TERMINATED BY ',';


LOAD DATA LOCAL INPATH 'C:/hadoop-lab/Hive/student.txt'
INTO TABLE student;

SELECT * FROM student;
```

Alter table:

```
ALTER TABLE student ADD COLUMNS (city STRING);
```

Create view:

```
CREATE VIEW student_view AS
SELECT name, age FROM student;
SELECT * FROM student_view;
```

View output

Ram 20
Shyam 22
Ravi 21

Create Function:

```
SELECT UPPER(name) FROM student;
```

Output:

RAM
SHYAM
RAVI

Create index:

```
CREATE INDEX idx_name
ON TABLE student (name)
AS 'COMPACT'
WITH DEFERRED REBUILD;
ALTER INDEX idx_name ON student REBUILD;
```

Drop operarions:

```
DROP VIEW student_view;

DROP TABLE student;

DROP DATABASE labdb;
```

9. Install, Deploy & configure Apache Spark Cluster. Run apache spark applications using Scala. Perform Data analytics using Apache Spark on Amazon food dataset, find all the pairs of items frequently reviewed together

Sample Dataset (food_reviews.csv)

```
U1,Milk
U1,Bread
U1,Butter
U2,Milk
U2,Bread
U3,Bread
U3,Butter
U4,Milk
U4,Butter
```

Install & Configure Spark

```
:: Set environment variables
set SPARK_HOME=C:\spark
set HADOOP_HOME=C:\hadoop
set PATH=%PATH%;%SPARK_HOME%\bin
```

Start Spark (Local Cluster Mode)

```
cd C:\spark\sbin
start-master.cmd
start-worker.cmd spark://localhost:7077
```

Scala Program (FrequentPairs.scala)

```
import org.apache.spark.sql.SparkSession

object FrequentPairs {
def main(args: Array[String]): Unit = {

val spark = SparkSession.builder()
.appName("Frequent Item Pairs")
.master("local[*]")
.getOrCreate()
```

```
val sc = spark.sparkContext

// Load data
val data = sc.textFile("C:/hadoop-lab/Spark/food_reviews.csv")

// Create (user, item)
val userItem = data.map(line => {
val parts = line.split(",")
(parts(0), parts(1))
})

// Group items by user
val grouped = userItem.groupByKey()

// Generate item pairs
val pairs = grouped.flatMap {
case (user, items) =>
val itemList = items.toList.distinct
for {
i <- itemList
j <- itemList
if i < j
} yield ((i, j), 1)
}

// Count pairs
val pairCounts = pairs.reduceByKey(_ + _)

// Filter frequent pairs (threshold = 2)
val frequentPairs = pairCounts.filter(_._2 >= 2)

// Print output
frequentPairs.collect().foreach(println)

spark.stop()
}
}
```

Compile & Run

```
cd C:\hadoop-lab\Spark

:: Compile
spark-shell -i FrequentPairs.scala
```

output:

```
((Milk,Bread),2)

((Bread,Butter),2)
```